

Lightning Web Components Style Guide

Table of Contents

1. [Style Vocabulary](#)
2. [When To Use LWCs](#)
3. [Basic Rules](#)
4. [Component Properties](#)
5. [Component Composition](#)
6. [Events](#)
7. [Component Styles](#)
8. [Testing](#)
9. [Documentation](#)
10. [Formatting](#)
11. [Resources](#)

Style vocabulary

This section is borrowed from the [Angular Style Guide](#), with some modifications, because we feel that these guideline recommendations are useful.

Each guideline describes either a good or bad practice. The wording of each guideline indicates how strong the recommendation is.

- **Do** is one that should always be followed. Always might be a bit too strong of a word. Guidelines that literally should always be followed are extremely rare. On the other hand, you need a really unusual case for breaking a Do guideline.
- **Consider** guidelines should generally be followed. If you fully understand the meaning behind the guideline and have a good reason to deviate, then do so. Please strive to be consistent.
- **Avoid** indicates something you should almost never do unless you have a good reason for doing otherwise.
- **Why?** gives reasons for following the previous recommendations.

None of these rules should be taken uncritically, however. This guide isn't designed to be used as a book to throw at someone, and if you have a reason to contradict any of the recommendations in here,

When To Use LWCs

Salesforce recommends that all new Lightning development is completed using Lightning Web Components, and we agree. If you're in a position where new Lightning component development is required, choose LWCs over Aura components. Converting Aura components to LWCs should be a case-by-case judgment call, depending on things like level of effort and ability. Overall, we strongly advise against writing new Aura components.

Basic Rules

Style

Take advantage of the fact that Salesforce transpiles your JavaScript and write the latest JavaScript wherever possible, such as: use `let` and `const` over `var`, use `class` rather than manipulate object prototypes directly, and so on. At the time of writing this, the JavaScript style guide is incomplete, so refer to the [Airbnb JavaScript style guide](#) in the meantime.

Components

Like Aura components, Lightning Web Components should follow the following rules:

- **Do** define components with one responsibility
- **Do** write as small components as possible
- **Do** strive to make your components reusable, where applicable
- **Avoid** using `style` attributes in tags, **do** create dedicated CSS files

Additionally, considering the nature of Lightning Web Components,

- **Consider** writing components that resemble HTML elements as much as possible.

Component Properties

- **Do** order your class properties and life cycle hooks in the following order:

1. Properties with `@api` decoration
2. Properties with `@track` decoration
3. Getters with `@api` decoration
4. Setters with `@api` decoration ¹
5. Properties with `@wire` decoration
6. `constructor`
7. `connectedCallback`
8. `disconnectedCallback`
9. `render`
10. `renderedCallback`
11. `errorCallback`
12. Methods with the `@api` decoration
13. Class methods

Why? This order resembles conventions within languages like Java; public properties come first, followed by protected and private properties, constructors, and methods.

- **Do** place decorators over the property or function they're supposed to decorate

```
export default Example extends LightningElement {  
  // bad  
  @api  
  bar;  
  
  // good  
  @api foo;  
  
  // bad
```

```
@api play() {  
    // ...  
}  
  
// good  
@api  
pause() {  
    // ...  
}  
}
```

- **Avoid** side effects in your computed properties

Why? Getters and setters in JavaScript behave like any other object attribute, and are expected to be referentially transparent.² By including side effects, they become harder to maintain and reason about.

```
import { LightningElement } from 'lwc';  
  
export default class ExampleComponent extends LightningElement {  
    name;  
    accessCount = 0;  
  
    // bad  
    @api  
    set name(value) {  
        // this is the side effect – we're modifying a value  
        // outside of the current scope  
        this.accessCount++;  
        this.name = value;  
    }  
  
    // good  
    @api  
    set name(value) {  
        this.name = value;  
    }  
}
```

- Constant values should be extrinsic to components

Why? It's more obvious that the value shouldn't change.

Note that unlike Apex, JavaScript isn't class-based, so the language allows you to have more than just a class definition in your files.

```
export default class BadExampleComponent extends LightningElement {  
    // bad  
    defaultName = 'component name';  
}
```

```

    @api name = this.defaultName;
  }

  const defaultValue = 'component name';

  export default class GoodExampleComponent extends LightningElement {
    // good
    @api name = defaultName;
  }

```

Component Composition

- **Avoid** component inheritance

Why? Doing so may cause snowballing complexity. Although the framework requires that we extend base LWC components, continuing to further extend these components leads to the same problems as inheritance in languages like Java. In general we should prefer [composition over inheritance](#), though "composition" in this case takes a slightly more literal meaning, meaning "your component's markup uses another component".

```

// bad, where `MyLightningElement` extends `LightningElement`
export default MyNewComponent extends MyLightningElement {
  // ...
}

```

- **Do** externalize shared code into modules

Why? There are three types of modules in LWC:

- "Private" modules exclusive to their component (in a sibling JS file for example)
- "Application" modules, which are utilized by application-specific components, but may not otherwise make sense for use outside of those components.
- "Generic" modules, which have broad applicability. They may contain generic, shared code for making writing JavaScript easier, or may contain some kind of utility class.

These names aren't hard and fast. Your "application" and "generic" modules are service components. You can further organize those modules into more specific files (say, for example, moving **Date** utilities into a separate file and exporting your functions), and specify exports at the root JS file.

```

// lwc/myCmp/myCmp.js

import { LightningElement } from 'lwc';
import { isRecordDisplayed } from './util';
import { specialSort } from './appModules';

export default class MyComp extends LightningElement {
  @api items;
}

```

```
    get displayedItems() {
        return this.items.filter(isRecordDisplayed);
    }

    get speciallySortedItems() {
        return this.items.sort(specialSort);
    }
}
```

```
// lwc/myCmp/util.js
// a "private" module

export function isRecordDisplayed(record) {
    return record.programmingLanguage !== 'JavaScript';
}
```

```
// lwc/appModules/appModules.js

// an "application" module, with code that could pertain to multiple
// components in the same application

import * as sortMethods from './sortMethods';

/*
    when it comes to service modules, something you can do if your API
    is very large is to import all non-default contents from sibling modules
    and re-export them as one module.
    that way, `specialSort` from `sortMethods` can be imported
    elsewhere like:

    import { specialSort } from './sortMethods';
*/

export { ...sortMethods };
```

```
// lwc/appModules/sortMethods.js

export function specialSort(left, right) {
    if (left.programmingLanguage === 'JavaScript') {
        return 1;
    } else {
        return -1;
    }
}
```

```
// lwc/genericModules/genericModules.js

// a "generic" module, with a function that could be used in any component
without context

// this function could be used with any Array
export function removeByIndex(array, index) {
    return [...array.slice(0, index), ...array.slice(index + 1)];
}
```

- **Consider** ordering your imports accordingly:

1. Absolute imports (e.g. 'lwc')
2. Absolute imports with paths (e.g. '@salesforce/wire-service-jest-util')
3. "Relative" imports (e.g. 'c/myModule')

Events

- **Avoid** mutating event data

Why? Except for primitive values (`number`, `string`, `boolean`, `bigint`, `null`, `undefined` and `symbol`), values are passed by reference and therefore **mutable**. This can introduce unpredictability and harm readability.

```
// bad

export default class Example extends LightningElement {
    compValue = [1, 2, 3];

    renderedCallback() {
        this.dispatchEvent('myevent', {
            detail: this.compValue
        });
    }
}

export default class ExampleHandler extends LightningElement {
    // bound to 'onmyevent'
    handleMyEvent(event) {
        // this changes `compValue` in `Example` component to [1,
2, 3, 4]
        event.detail.push(4);
    }
}
```

```
// good

export default class Example extends LightningElement {
  compValue = [1, 2, 3];

  renderedCallback() {
    // this clones the array, breaking the reference
    // it's the same as `this.compValue.slice()`
    this.dispatchEvent('myevent', {
      detail: [...this.compValue]
    });
  }
}
```

- **Consider** descriptive event handler names

Why? It's helpful to know what event names and handlers are doing.

```
export default class Example extends LightningElement {
  // not preferable – doesn't describe what handler is doing
  handleClick(event) {
    updateFields();
  }

  // preferable – indicates what happens when event is handled
  updateFieldsOnClick(event) {
    updateFields();
  }
}
```

Component Styles

Styles in LWC are encapsulated within their corresponding components. All the standard CSS best practices apply, but do note that the `:host` directive is roughly the same as `.THIS` in Aura.

Testing

Lightning Web Components use the [Jest](#) testing framework.

- **Do** write tests!

Why? Writing tests allows you to automatically verify the correctness of your code. This goes without saying. Jest tests in particular have more usefulness than just providing code coverage, however, as they can serve as unit tests, integration tests, and even UI tests. Although as of writing this Salesforce doesn't require that you write tests, unlike Apex. Your codebase will be stronger with them, and we strongly encourage you to do so. We also encourage you to write useful tests.

- **Do** organize your test suites around components

Why? By testing components, you can test the functionality of your component with a great degree of granularity. Generally, you should use **describe** for a component, and **it** for specific tests for that component. This produces a nice, readable test output. Larger test suites may have child **describes**.

```
describe('myComponent', () => {
  it('should display a table upon rendering', () => {
    // ...
  });

  it('should format records passed into it', () => {
    // ...
  });
});
```

- **Avoid** testing multiple components in one test suite

Why? Similar to the rule above, tests should correspond to a component 1:1. Your tests should be isolated so that you can reliably test any single component

- **Do** mock dependencies

Why? Jest has **strong mocking capabilities**. Simply put, you're writing tests for your components, not the things it depends on. Things worth mocking:

- Child components
- Callouts
- **@wire** methods
- Code from other modules that may perform callouts or some other expensive behavior

See the [documentation](#) for mocking specifically in LWC.

- **Consider** writing component tests according to requirements, or at least behavior

Why? This is just good practice when it comes to testing. Jest uses a very conventional JavaScript test suite pattern born out of something called behavior-driven development (BDD), and is well suited for writing BDD tests.

Documentation

At the time of writing this, LWC doesn't support documentation in the same way that Aura does. One workaround you could use is **JSDoc**. JSDoc is usually used to generate web-based documentation for APIs and libraries, although some IDEs are smart enough to provide auto-completion or other contextual information when you specify JSDoc for your functions and classes.

```
// inside of some service module...

/**
 * Adds two numbers.
```

```
* @param a {number} left-hand side of addition
* @param b {number} right-hand side of addition
* @return {number} the sum of `a` and `b`.
* @example
*
*         const sum = add(2, 3); // => 5
*/
export function add(a, b) {
    return a + b;
}
```

Formatting

Things like spaces vs. tabs, tab width, and so on, are trivialities that can bog down a developer. What's important is that a convention is used and used consistently.

For both HTML and JavaScript, LWCs are supported by the excellent JavaScript formatter [Prettier](#).

Resources

- [Airbnb JavaScript style guide](#). Since we don't have a purely JavaScript style guide at the time of writing this, this should serve as a good stand-in.
- [Salesforce's LWC developer documentation](#)
- [LWC Recipes](#). Please note that Salesforce themselves may contradict some of the rules stated in this style guide.
- [Jest JS](#)
- [Salesforce's repo for Jest testing in LWC](#)
- [Jest Test Patterns and Mock Dependencies](#)

-
1. Although it's possible to have setters with the `@api` decorator, [Salesforce recommends getters be decorated rather than setters](#). ^
 2. An expression is said to be referentially transparent when, essentially, an expression evaluates to the same result in any context. If a function is referentially transparent, it is also pure. ^